

Weight and Activation Precision:

Quantized Models are often described using $W \times A$ notation, where W gives the weight bit width and A gives the activation bit width. Using different formats for different model components is a form of mixed-precision inference. For example, $W8A16$ is widely used because weights can use INT8 to reduce storage + memory traffic while activations remain FP16 since values change every input and can't be represented accurately with integers.

Notation	Common Interpretation
$W8A16$	INT8 weights with FP16 or BF16 activations
$W8A8$	INT8 weights and activations or FP8 weights & activations
$W4A16$	INT4 weights with FP16/BF16 activations

So a system could use:

Weights: INT8

Activations: FP16

KV Cache: FP8

Softmax: FP32

The notation alone doesn't specify the specific format, it only provides the bit width (eight-bit, 16-bit, etc.)

Section 4: Why fewer bits can help

During inference, a model's learned parameters are normally loaded into system/accelerator memory and repeatedly read during execution

FP32 $\rightarrow$ ~ 4 GB of storage

FP16/BF16 $\rightarrow$ ~ 2 GB of storage

INT 8 $\rightarrow$ ~ 1 GB of storage

INT 4 $\rightarrow$ ~ 0.5 GB of storage

Storage Size
for a model
with one
billion parameters

KV cache is created separately during inference, & stores keys and values from previously processed tokens. As the cache grows with sequence length, batch size, and model architecture, quantizing the KV Cache reduces this dependent memory, but this is separate from the one billion parameter calculation above

A simple breakdown is:

Total Active Inference Memory = parameters + KV Cache + temporary activations + runtime workspaces

Connecting quantization to memory hierarchy

CSGIC introduces us to the memory hierarchy:

Storage $\rightarrow$ DRAM $\rightarrow$ cache/SEAM $\rightarrow$ registers $\rightarrow$ compute units

SLOWER and LARGER

FASTER and SMALLER

Before a processor can operate on model parameters, the required data must reach its compute units, but moving all this data can take up a lot of time and energy

Let's say we have a 64-byte cache line. It can hold either 16 FP32 parameters or 64 INT8 parameters. The same cache line holds 4x as many INT8 parameters. So smaller representations can signify $\rightarrow$

1. Less storage for the model
2. Less data transferred from DRAM
3. Less energy spent moving/transferring data

However, a smaller representation DOES NOT automatically lead to faster execution

The runtime must be able to use the representation efficiently, some costs include:

- Unpacking: Extracting multiple low-bit values from a packed byte/word
- Dequantization: scaling to reconstruct values
- Padding & Alignment: Storing extra unused values to make sure it supports hardware-layout requirements
- Format Conversion: converting unsupported values to FP16, BF16, FP32, etc.

FP32:
 DRAM $\rightarrow$ $\underbrace{[32][32]}_{\text{fewer parameters}} \rightarrow \text{COMPUTE}$

INT 8:
 DRAM $\rightarrow$ $\underbrace{[8][8][8][8][8][8][8][8]}_{\text{more parameters}} \rightarrow \overset{\text{overhead} \text{ ;}}{[\text{UNPACK} + \text{SCALE}]} \rightarrow \text{COMPUTE}$

Overall, quantization improves performance only when the saved memory movement outweighs this overhead. So quantization is therefore a hardware - software tradeoff, not just a way to shrink a model's data!